payTrace 2.0

Real-Time Payment Fraud Detection

Project report: overview, objectives, architecture, methodology, tech stack and results

Event	Datanovathon 2026
Problem	DN-FN-01 · Real-Time Payment Fraud Detection (Finance, Advanced)
Deliverable	Working web application (customer, analyst and admin views), REST scoring API, trained models and evaluation
Data	Sparkov card payments (1.85M, Kaggle) · IEEE-CIS real Vesta payments (590k, Kaggle) · synthetic v2 data
Date	7 November 2026

All results in this report were measured on each dataset's held-out final period, never used for training or tuning. Sparkov is simulated data and IEEE-CIS is real but anonymised; results on a bank's own data will differ.

1. Project overview

Digital payments in India move in seconds, and fraud has to be stopped in that same window: once a UPI or card payment completes, the money is usually gone. payTrace checks every payment **before it completes** and returns one of three decisions in a few milliseconds:

Decision	When	What happens
ALLOW	Normal for this customer	The payment is sent immediately.
REVIEW	Moderate risk: something is unusual	The customer sees what looked different and decides: send it after a CAPTCHA security check, or cancel.
BLOCK	High risk: strong fraud signs together	The payment is blocked with no customer override. It goes to the fraud team's case queue.

It combines a supervised model that recognises known fraud with anomaly detectors that catch fraud it has never seen, wraps them in bank policy rules, and explains every decision in plain language: to customers with real numbers (for example “₹70,000 is about 10.5 times your usual payment of ₹6,694”), and to analysts with reason codes, a risk breakdown and an AI-written case analysis.

The system is a complete web application with three roles and an API:

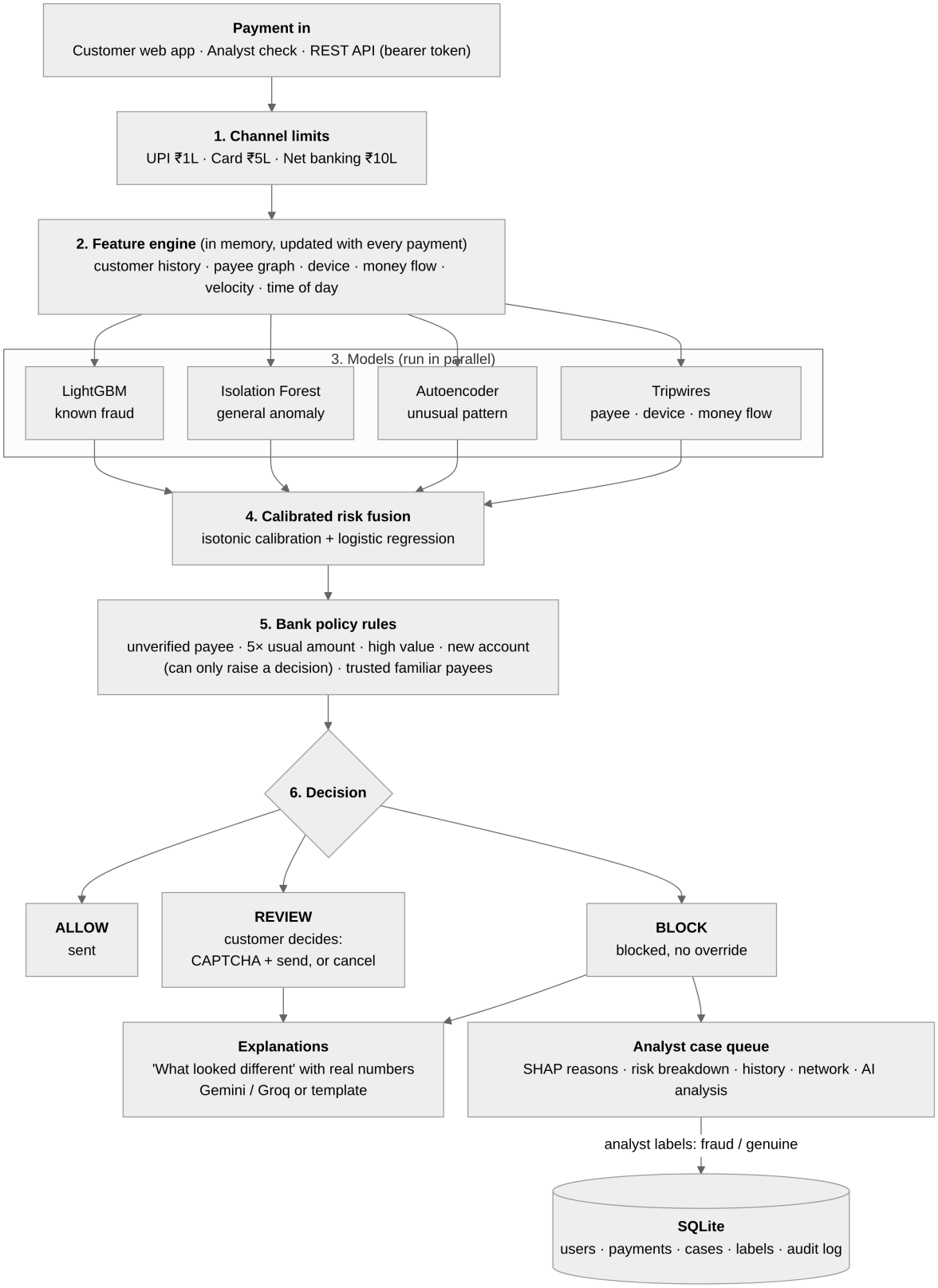
User	What they do
Customer	Sends money from a banking-style screen and sees each decision with an explanation; chooses on moderate-risk payments; can view activity and sign up from the website.
Fraud analyst	Works a prioritised case queue with reasons, history, similar cases, payment network and AI analysis; marks payments fraud or genuine; checks payments for customers.
Admin	Live KPIs, simulated traffic, real-data replay, model evidence, user management, LLM connection test, audit log.
Other systems	POST /api/v1/score with a bearer token returns the decision, risk and reasons.

2. Objectives

#	Objective	How it is met
1	Decide in real time	In-memory feature engine and pre-loaded models; app payments are checked in about 3–6 ms end to end.
2	Catch known *and* new fraud	LightGBM for labelled patterns; Isolation Forest, autoencoder and payee/device/money-flow tripwires for anomalies.
3	Keep false alarms low	Calibrated risk; thresholds chosen on a separate period to minimise cost; trusted familiar payees; customer choice instead of a hard stop on moderate risk.
4	Explain every decision	SHAP reason codes, risk breakdown, plain-language customer messages, LLM-written analysis grounded only in evidence.
5	Prove it on real data	Retrained and evaluated on Sparkov and real IEEE-CIS data with strict time-based splits; transfer and pooling experiments reported honestly.
6	Be safe and usable	Role-based access, hashed passwords, CSRF protection, CAPTCHA, rate limits, audit log, secrets only in .env.

3. Architecture

Every payment, whether typed in by a customer, checked by an analyst or sent by another system through the API, goes through the same pipeline:



Components

Layer	What it does
Channel limits	Declines payments above the channel limit before any scoring (UPI ₹1,00,000; card ₹5,00,000; net banking ₹10,00,000; configurable).
Feature engine	Keeps each customer's, payee's and device's recent history in memory and updates it with every payment, so features are always current. The dataset's later traffic is replayed up to the app's clock, so the bank never looks silent.
Models	Four detectors scored in parallel (section 4.3).
Risk fusion	Combines the detectors into one calibrated probability with visible weights.
Policy rules	Deterministic bank rules that can only raise a decision, never lower it; familiar payees are trusted for similar amounts.
Decision and explanation	ALLOW / REVIEW / BLOCK, with plain-language reasons for customers and full evidence for analysts.
Storage	SQLite (WAL mode): users, payments, cases, analyst labels, audit log. Models and engine state saved with joblib/pickle for a ~4 s start.
Web app and API	Starlette + Uvicorn server, single-page JavaScript front end, bearer-token scoring API.
LLM (optional)	Google Gemini (free tier), Groq as backup. Only rewords verified evidence; template fallback if absent or slow.

4. Methodology

4.1 Data

Dataset	Source	Size	Fraud rate	Used for
Sparkov	Kaggle (kartik2112), simulated card payments, CC0	1,852,394 payments · 999 customers · 693 merchants · 2019–2020	0.52%	Web app model
IEEE-CIS	Kaggle, real Vesta card-not-present payments, anonymised	590,540 payments · 431 columns · Dec 2017–Jun 2018	3.50%	Real-data model and replay
Synthetic v2	Our generator (UPI-style, injected fraud patterns)	277,229 payments · 5,242 customers	1.48%	Baseline, design experiments

Adapters convert each dataset to one common schema (customer, payee, amount, category, channel, device, city, time). Amounts are converted to rupees at a fixed ₹88 per US dollar. For IEEE-CIS, a customer id is built from card and address fields (card1 + addr1 + account-start day).

4.2 Leakage-free time splits

Each dataset is split by time, never at random, so the model is always tested on the future:

Train	Early stopping	Calibration	Threshold choice	Test (reported)
0–50%	50–60%	60–67%	67–75%	75–100%

Features for a payment use only what happened before it. Fraud labels feed graph features only after a 3-day delay, as chargebacks arrive late in reality. The test period is never used for training, calibration or thresholds.

4.3 Features and models

Features describe how this payment compares with the customer's own behaviour and the network around it: amount versus the customer's usual and largest amounts, time of day, velocity (payments in the last minutes and hours), first payment to this payee, new device or city, payee popularity and recent confirmed fraud at the payee or device, and money flow (money received and sent on quickly).

Model	Role	Why
LightGBM (gradient-boosted trees)	Main supervised model	Learns labelled fraud patterns; fast; gives exact SHAP reason codes.
Isolation Forest	General anomaly score	Flags payments unlike anything normal, even without labels.
Autoencoder (scikit-learn MLP)	Pattern anomaly	High reconstruction error means an unusual combination of features.
Tripwires (3 Isolation Forests)	Payee, device and money-flow anomalies	Each can send a payment to REVIEW on its own, catching new scams.
Isotonic calibration + logistic fusion	One calibrated risk	Probabilities that mean what they say; fusion weights are visible in the breakdown.

4.4 Decision thresholds

Thresholds are set on the 67–75% period, not by a fixed 0.5. **REVIEW** minimises expected cost (the money lost to missed fraud plus ₹100 for each false alarm). **BLOCK** is the lowest risk at which 95% of blocked payments are really fraud, so blocking is reserved for near-certain cases.

4.5 Bank policy layer

A model only knows what it was trained on; it cannot tell that a payee doesn't exist. So, as in a real payment system, rules sit around it and can only raise a decision:

- Unverified payee → REVIEW; unverified payee with ₹50,000 or more → BLOCK.
- High value (₹2,00,000 or more), 5× the customer's usual amount (and ₹25,000 or more), or a large payment from a new account → REVIEW.
- 10× the customer's largest past payment from a new device or to an unverified payee → BLOCK; three or more warning rules at once → BLOCK.
- **Familiar payees:** a payee the customer has already paid (bank history or confirmed in the app) is trusted for up to twice the largest amount paid to them. For these, an anomaly flag on its own no longer stops the payment; the model's risk and the rules still apply.

4.6 Customer decision flow

- **REVIEW:** the customer sees “What looked different”, “What you can do” and “How we checked this payment”, then chooses *Yes, send it* (after a CAPTCHA) or *Cancel*.
- **BLOCK:** no choice is offered and no money leaves the account; only an analyst can release it after investigating.
- **CAPTCHA:** self-hosted SVG, 5 characters mixing capital letters, small letters and digits (case matters, look-alikes such as 0/O/o and 1/I/l removed), light noise, single use, expires in 10 minutes, 5 tries.

4.7 Explainability and the AI assistant

Every flagged payment carries SHAP reason codes (from LightGBM) and a risk breakdown showing how each detector moved the risk from the starting point. For customers, the bank computes real comparisons (amount versus usual, first payment to a payee, late-night timing, new device), and the LLM writes them as a short message; it never sees or shows model scores or thresholds. For analysts, the LLM writes a structured case analysis (risk assessment, evidence for and against, policy rules, network context, similar cases, what would change the assessment, next steps) from an evidence pack only. The LLM never makes a decision.

4.8 Feedback loop

Analysts label payments as fraud or genuine. Labels feed the graph features (with delay) and a champion/challenger retraining step: a retrained model is promoted only if it beats the current one on recent data.

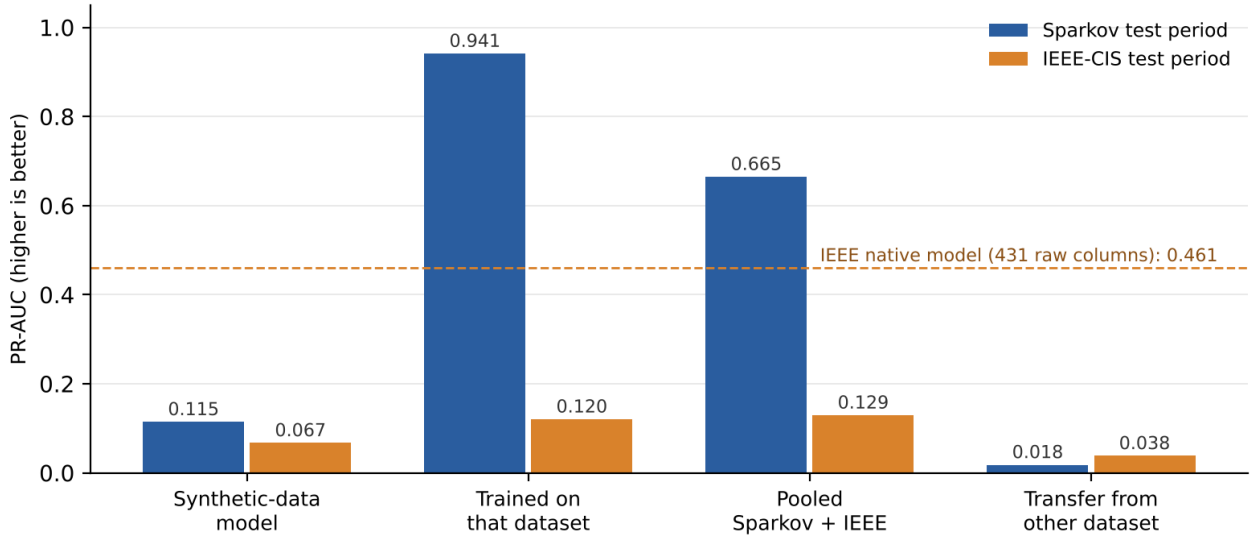
5. Tech stack

Area	Technology
Language	Python 3
Data	pandas, NumPy, PyArrow / Parquet
Machine learning	LightGBM (main model, SHAP via pred_contrib); scikit-learn (Isolation Forest, MLP autoencoder, isotonic calibration, logistic regression); XGBoost and CatBoost for comparison only
Graph analysis	NetworkX (payee/device network, Louvain clusters, money chains)
Web server and API	Starlette + Uvicorn (async), JSON REST API, bearer token for systems
Front end	Single-page app in plain HTML, CSS and JavaScript (no framework), SVG CAPTCHA
Database	SQLite in WAL mode (PostgreSQL path available)
Security	PBKDF2 password hashing, HMAC-signed session cookie, CSRF header check, role-based access, rate limits, audit log, secrets in .env
AI / LLM	Google Gemini API (free tier, automatic model discovery), Groq as backup, template fallback
Streaming	In-process event bus with dedupe, re-ordering and dead-letter queue; Kafka/Redpanda adapter (optional)
Analytics dashboard	Streamlit + Plotly (admin, detailed charts)
Model storage	joblib, pickled engine state, model registry with champion/challenger roles
Testing	pytest + httpx (61 automated tests); Playwright for browser checks
Deployment	run_app.sh (one command), Docker Compose

6. Results

6.1 Does the model work on data it wasn't designed on?

PR-AUC (area under the precision-recall curve) is the main measure because fraud is rare: a model that flags nothing still scores 99% accuracy, but PR-AUC stays near the fraud rate. All numbers are on each dataset's unseen test period.



Model	Sparkov PR-AUC	IEEE-CIS PR-AUC
Synthetic-data model (original)	0.115	0.067
Trained on that dataset alone (common features)	**0.941**	0.121
Pooled Sparkov + IEEE-CIS	0.665	0.129
Transfer: trained on the other dataset	0.018	0.038
IEEE-CIS native model (431 raw columns)	–	**0.461**
IEEE-CIS native + our behaviour features	–	0.394

Findings. The model trained on synthetic data failed on both real-style datasets, which is why we retrained. A model must be trained on data like the bank's own: transfer between datasets fails, and pooling two different datasets costs accuracy on the cleaner one. On IEEE-CIS, Vesta's own 431 anonymised columns carry most of the signal, so the native model is far better than our common features.

6.2 The web app's model (Sparkov, unseen last 6 months)

Measure	REVIEW tier	BLOCK tier
Fraud caught (recall)	**98.8%**	83.8%
Flagged payments that are fraud (precision)	18.3%	**92.2%**
Genuine payments flagged (false-alarm rate)	1.70%	–
Fraud value caught	98.8%	–
ROC-AUC / PR-AUC	0.998 / 0.941	
Training time	about 1.5 minutes	

Out of 523,388 test payments with 2,008 frauds, the model flags almost all fraud while asking about fewer than 2 in 100 genuine payments, and its blocks are right more than 9 times in 10.

6.3 Real fraud: IEEE-CIS native model (unseen last 25%)

Measure	Value
PR-AUC / ROC-AUC	0.461 / 0.862
REVIEW tier	70% of fraud flagged, 13.7% of genuine payments flagged
BLOCK tier	8.2% of fraud blocked, 98.1% of blocks correct
Test set	128,823 real payments, 4,393 frauds

Real fraud is much harder: catching 70% costs many more false alarms than on simulated data. The admin **Real data** page replays these payments live and keeps a running score of caught, missed and false alarms against the true labels.

6.4 Application behaviour

Scenario (demo customers; usual payment about ₹6,694)	Result
₹4,000 / ₹9,000 / ₹23,000 to a usual merchant	ALLOW: sent
₹23 to a merchant the customer has paid ₹59,132 before	ALLOW (familiar payee; automated test)
₹70,000 to a usual merchant (10.5× usual)	REVIEW: customer decides with CAPTCHA
₹23 to a payee confirmed earlier with the CAPTCHA	ALLOW
₹90,000 to an unknown payee	BLOCK: no customer override
Several payments within seconds	REVIEW (burst of payments)

Checks take about 3–6 ms per payment in the app. On the synthetic benchmark, the scorer alone ran at 0.46 ms median and 3.2 ms at the 99th percentile (about 1,850 payments per second per core).

7. Responsible AI and security

- **Decision boundary:** payTrace recommends; the LLM has no path to any decision.
- **Explainability:** every alert shows reasons, the risk breakdown and which detectors fired.
- **Human in the loop:** blocked payments are released only by analysts; contradicting labels need a note and a second analyst.
- **Privacy:** customers never see model scores or thresholds; free-tier LLMs only receive simulated data here.
- **Access:** customer, analyst and admin roles; customers sign up on the website; only admins create staff accounts.
- **Fairness:** false-alarm rates are reported by city, channel and account age; new accounts get step-up checks, not blocks.

8. Limitations and next steps

- Sparkov is simulated and IEEE-CIS is anonymised; a bank's own data is needed for production performance, and real labels arrive late and can be wrong.
- A model must be retrained per bank: transfer between datasets failed in our experiments.
- The CAPTCHA proves a person is answering, not who it is; a real deployment should add a one-time code to the registered phone.
- Device and location are chosen in demo controls; a real app reads them from the phone.
- Kafka and PostgreSQL paths exist but were not run at scale; the tested path is the in-process bus with SQLite.
- Next: phone OTP, real device fingerprinting, GNN experiments on the payment graph, and monitoring for model drift.